

Manual das bibliotecas

Capítulo 1 - Biblioteca Botao.h	2
O tipo Botao:	2
I. Estrutura "botao":	2
II. Função "inicializa_botao":	2
III. Exemplo de utilização da biblioteca:	2
a) Declaração:	2
b) Inicialização:	2
c) Utilização:	3
O tipo Botao_filtrado:	3
I. Estrutura "Botao_filtrado":	3
II. Função "botao_filtrado_clicado":	3
III. Exemplo de utilização da biblioteca:	4
d) Declaração:	4
e) Inicialização:	4
f) Utilização:	4
Capítulo 2 - Biblioteca Temporizador.h	5
O tipo Temporizador	5
I. Estrutura Temporizador	5
II. Função inicializa_temporizador	5
III. Exemplo de utilização da biblioteca	5
g) declaração	5
h) inicialização	5
i) utilização	5
Capítulo 3 - Biblioteca Utilidades.h	6
Função "set_frequency":	6
Função get_adc_cal":	6
Função setup_tick":	7
j) utilização	7
Botão filtrado com tempo desejado de 100 ms para filtrar o ruído.	7
Temporizador com tempo desejado de 500 ms.	7
Observações e recomendações:	7

Capítulo 1 - Biblioteca Botao.h

A biblioteca “Botao.h” define estruturas e funções para lidar com botões em microcontroladores. Ela tem dois tipos de botões: um sem filtro(Botao) e outro com filtro de oscilações(Botao_filtrado). Será explicado as principais funções e estruturas:

O tipo Botao:

Deve ser utilizado para detectar borda de subida ou de descida quando o sinal **não** apresenta **ruído** nas transições.

I. Estrutura “botao”:

- Definição de uma estrutura “botao” que representa o estado de um botão.
- Possui os seguintes campos:
- “subida”: registra o tipo de borda, se por subida ou por descida. Se for 1, a borda será de descida, se for 0, a borda será de subida. .
- “inverter”: variável interna que controla a borda.
- “f1”: variável interna do estado anterior do botão.

```
typedef unsigned char byte;
struct botao { //definição do meu novo tipo de dado temporizador
    char subida;
    char inverter;
    char f1;
};
// fim da definição da estrutura
```

II. Função “inicializa_botao”:

- Inicializa a estrutura “Botao”.
- Recebe como argumentos o tipo de borda (1 para subida e 0 para descida”) e o endereço do objeto do tipo “Botao”.

```
void inicializa_botao(byte subida, Botao* b);
```

III. Exemplo de utilização da biblioteca:

Deseja-se um botão sem filtro para detectar borda de subida de um sinal sem ruído para incrementar um contador.

a) Declaração:

Botao br;

b) Inicialização:

- Subida = 1.
- O endereço de br deve ser passado como segundo argumento.

```
inicializa_botao(1,&br);
```

c) Utilização:

Função “botao_clicado”:

- Verifica se o botão foi clicado.
- Recebe como argumentos o endereço do botão e o estado atual do botão (“entrada”).
- Retorna 1 se o botão foi clicado e 0 se não foi clicado.

Protótipo da função:

```
byte botao_clicado(Botao* b, byte entrada); // entrada tem que ser 0 ou 1
```

Exemplo da utilização da função:

- O endereço de br é passado para a função botao_clicado.
- e o estado do pino P2.1

```
if (botao_clicado(&br, (P2IN & BIT1)? 1: 0) ){ // botão no pino p2.1 foi
//clicado?
    P2OUT ^= BIT0; // se sim, comuta o estado da P2.0 quando botão for clicado
}
```

O tipo Botao filtrado:

Deve ser utilizado para detectar borda de subida ou de descida quando o sinal apresenta **ruído** nas transições.

I. Estrutura “Botao_filtrado”:

- Definição da estrutura “Botao_filtrado” para filtrar oscilações.
- Possui os seguintes campos:
- “t”: Temporizador para controlar a filtragem de oscilações.
- “b”: o tipo de dado “Botao”.
- “osc”: Indica se houve oscilação do estado do botão.
- “ent_ant”: Estado anterior do botão.

```
struct botao_filtrado {
    Temporizador t;
    Botao b;
    char osc;
    char ent_ant;
};
```

II. Função “botao_filtrado_clicado”:

- Verifica se o botão filtrado foi clicado, evitando oscilações indesejadas.
- Recebe como argumentos o endereço do “Botao_filtrado” e o estado atual do botão (“entrada”).
- Retorna um valor indicando se o botão filtrado foi clicado.

```
char botao_filtrado_clicado(Botao_filtrado* bf, byte entrada);//retorna se o botao filtrado foi clicado
```

III. *Exemplo de utilização da biblioteca:*

Deseja-se um botão com filtro para detectar borda de subida de um sinal sem ruído para incrementar um contador.

d) Declaração:

Botao_filtrado bf;

e) Inicialização:

- Inicializa a estrutura “Botao_filtrado”, configurando o temporizador para filtragem de oscilações.
- Recebe como argumentos o estado inicial do botão (“subida”), o endereço do botão filtrado do tipo “Botao_filtrado” e o tempo de oscilação a ser considerado (“tempo_oscilacao”).

```
void inicializa_botao_filtrado(byte subida, Botao_filtrado* bf, byte tempo_oscilacao);  
//Exemplo:  
inicializa_botao_filtrado(1, &bf, 10);//configura para filtrar em 100 ms
```

f) Utilização:

Função “botão_clicado”:

- Subida = 1.
- O endereço de bf do tipo "botão_filtrado" será passado para a função.
- Tempo_oscilacao = 10 (100 ms para o TEMPO_TICK em 10.000 us, para $f_{SMCLK} = 1$ MHz).

Protótipo da função:

```
byte botao_filltrado_clicado(Botao* b, byte entrada);//entrada tem que ser 0 ou 1
```

Exemplo da utilização da função:

- Verifica se o botão do tipo filtrado foi clicado.
- Recebe como argumentos o endereço do botão e o estado atual do botão (“entrada”).
- Retorna 1 se o botão foi clicado e 0 se não foi clicado com segurança mesmo com a existência de ruídos.

```
if (botao_filtrado_clicado(&bf, (P2IN & BIT2))){//verifica se o botão no pino p2.2 foi clicado  
    P2OUT ^= BIT0;  
}
```

Capítulo 2 - Biblioteca Temporizador.h

A biblioteca “Temporizador.h” define uma estrutura e algumas funções relacionadas a temporizadores.

O tipo Temporizador

Deve ser utilizado para aguardar um certo definido pelo usuário para realizar alguma tarefa no código.

I. Estrutura Temporizador

- Define uma estrutura “temporizador” que representa um temporizador.
- Possui os seguintes campos:
- “ti”: tempo inicial do temporizador.
- “tempo”: Tempo desejado com base no tempo do tick (unidade de tempo específica do sistema).
- “to”: Indica se o temporizador chegou ao tempo desejado.

```
struct temporizador { //definição do meu novo tipo de dado
temporizador
    unsigned int ti;
    unsigned int tempo; //tempo desejado com base no tempo do tick
    char to;
}; // fim da definição da estrutura
```

II. Função inicializa_temporizador

- Inicializa a estrutura “Temporizador” com um tempo desejado.
- Recebe como argumentos o tempo desejado (“tempo”) e um ponteiro para a estrutura “Temporizador”.

```
void inicializa_temporizador(unsigned int tempo, Temporizador* t);
```

III. Exemplo de utilização da biblioteca

Deseja comutar o estado do pino P2.4 e P2.5 a cada 500 ms e 100 ms respectivamente. Para isso serão necessários dois temporizadores. Um para o tempo de 500 ms e outro para 100 ms.

g) declaração

```
Temporizador t1, t2; //declarados os temporizadores t1 e t2
```

h) inicialização

```
inicializa_temporizador(500,&t1); //incializando t1 com 500 ms
inicializa_temporizador(100,&t2); //incializando t2 com 100 ms
```

i) utilização

Função “passou_tempo”:

- Verifica se o tempo desejado foi atingido pelo temporizador.
- Recebe como argumento o **endereço** do “Temporizador”.
- Retorna 1 indicando se o tempo desejado foi atingido, 0 caso contrário.

```
char passou_tempo(Temporizador* t);
```

Exemplo da utilização da função:

```
if (passou_tempo(&t1)){//aguarda 500 ms p/ inverter o estado do bit4 da porta2
    P2OUT ^= BIT4;           //inverte o estado do bit 4 da porta 2
    reseta_temporizador(&t1); //reseta a contagem de tempo de t1
}

if (passou_tempo(&t2)){//aguarda 100 ms p/ inverter o estado do bit5 da porta2
    P2OUT ^= BIT5;           //inverte o estado do bit 5 da porta 2
    reseta_temporizador(&t2); //reseta a contagem de tempo de t2
}
```

Função “reseta_temporizador”:

- Reseta o temporizador para que aguarde o tempo programado novamente. Recebe como argumento o endereço do temporizador do tipo “Temporizador”.

```
void reseta_temporizador(Temporizador* t);
```

Capítulo 3 - Biblioteca Utilidades.h

Este arquivo de cabeçalho (“Utilidades.h”) define as funções que podem ser úteis em um sistema. Explicaremos cada uma delas:

Função “set frequency”:

Esta função recebe a frequência em MHz (1, 8, 12 ou 16) como entrada e o endereço do vetor de configuração do DCO (Oscilador Controlado Digitalmente). Ela é responsável por configurar a frequência do sistema de acordo com o valor fornecido. O vetor `vetor\_dco` é usado para armazenar as configurações do DCO após a função ser executada.

```
void set_frequency(unsigned char freq_MHz, unsigned char* vetor_dco);
```

Função get adc cal”:

Esta função recebe uma constante como entrada e o endereço do vetor de calibração do conversor analógico-digital (ADC). Ela é responsável por obter o valor calibrado do ADC com base na constante fornecida. O vetor “vetor_adc” é usado para armazenar os valores de calibração do ADC.

```
unsigned int get_adc_cal(unsigned int constante, unsigned int* vetor_adc);
```

Função setup_tick”:

Esta função recebe o tempo de tick como entrada e é responsável por configurar o tempo do tick do sistema. O tempo do tick precisa ser informado para que haja o funcionamento correto da biblioteca do temporizador e do botão do tipo filtrado e para outras finalidades futuras.

```
void setup_tick(unsigned int tempo_tick);
```

j) utilização

Exemplo de utilização da função:

```
#define TEMPO_TICK 10000 //us, para fSMCLK em 1 MHz  
setup_tick(TEMPO_TICK); //informa o tempo do tick
```

Fórmula do cálculo do tempo em função do tempo do *tick*:

"Tempo" = Tempo_desejado/(TEMPO_TICK); Sendo TEMPO_TICK em us e o Tempo_desejado em segundos. Esse "Tempo" deve ser passado para a inicialização do botão_filtrado no argumento "**tempo_oscilacao**" e "**tempo**" no caso da inicialização do temporizador.

Exemplo: TEMPO_TICK de 10.000 µs.

Botão filtrado com tempo desejado de 100 ms para filtrar o ruído.

"Tempo" = Tempo_desejado/(TEMPO_TICK) = 100 ms/(10.000 µs) = **10**; Deve ser passado para a função esse valor 10 da seguinte forma:

Temporizador com tempo desejado de 500 ms.

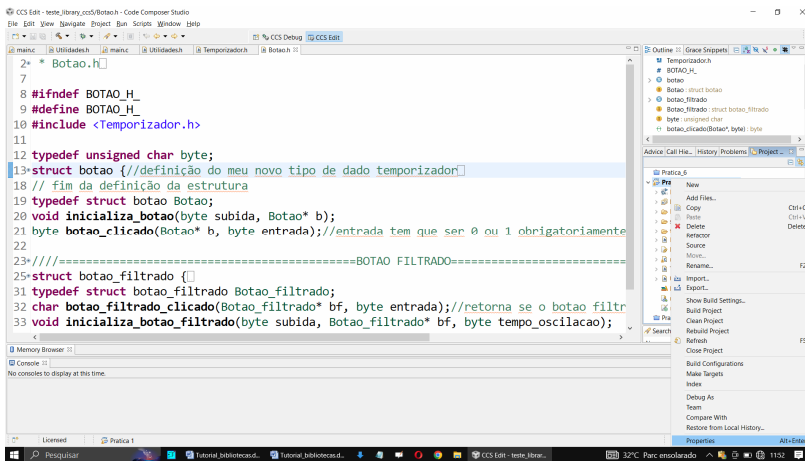
"Tempo" = Tempo_desejado/(TEMPO_TICK) = 500 ms/(10.000 µs) = **50**; Deve ser passado para a função esse valor 10 da seguinte forma:

```
inicializa_temporizador(50,&t); //temporizador está inicializado com 500 ms
```

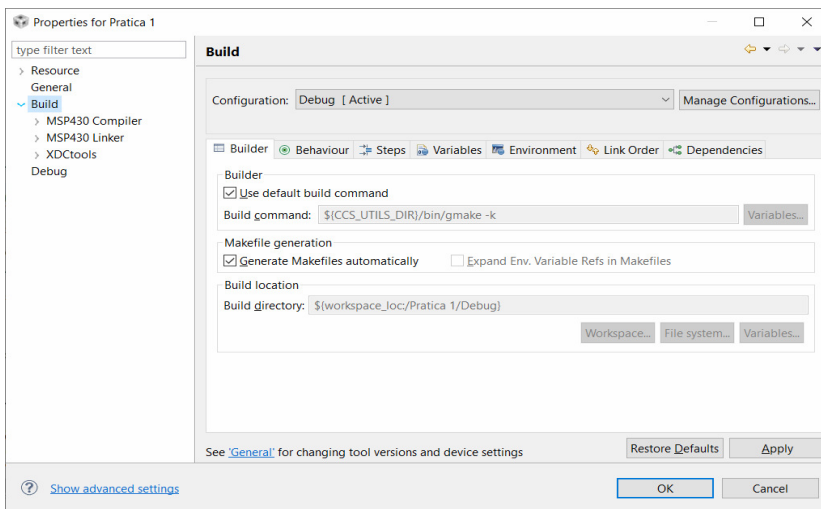
Observações e recomendações:

Deve-se incluir a biblioteca desejada no seu arquivo fonte, o arquivo de extensão .h na pasta do projeto bem como o arquivo "Library_release.lib". Deve-se também configurar no CCS o caminho do arquivo cabeçalho da biblioteca conforme ilustrado abaixo:

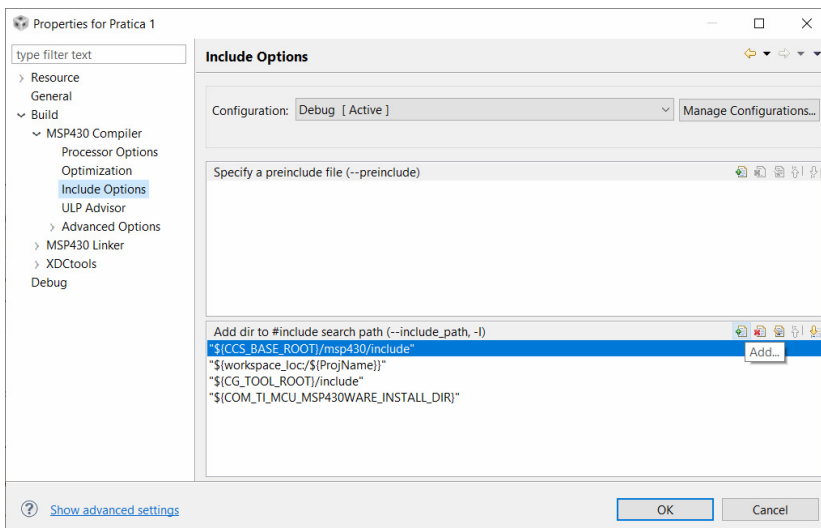
Aponte o cursor do mouse para a pasta do projeto e click com o botão direito do mouse. Na última opção, click em "Properties".



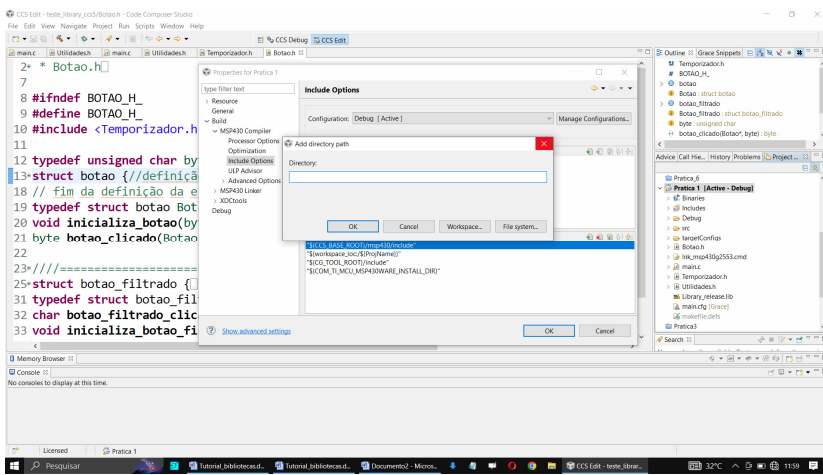
Click em ">" da opção "Build"



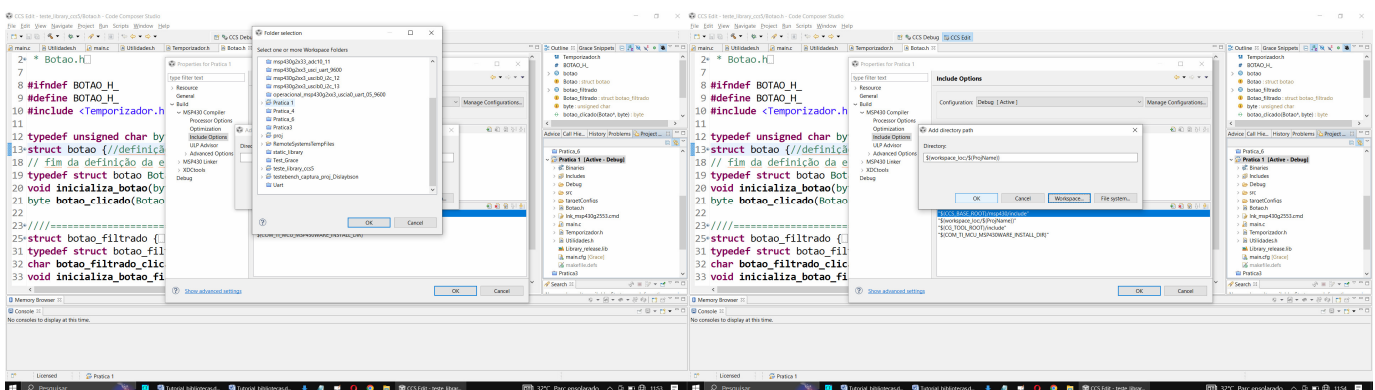
Click em ">" da opção "MSP430 Compiler", click em "Include Options" e click no botão "add" como na figura.



Click em "Workspace..."



Click na pasta do seu projeto e então em ok e ok novamente.



Ao utilizar a biblioteca, deve-se sempre primeiro **D**eclarar o objeto desejado, em seguida **I**nicializá-lo e depois **U**tilizá-lo (**DIU**).